Quiz — 2.1.5 Thinking Concurrently — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Section A (6 marks — 1 each) 1. **B** — independent regions processed simultaneously on multiple cores is parallel/concurrent processing. 2. **B** — scene 1 and scene 2 audio are independent (no shared data), so they can run together; the others have dependencies. 3. **B** — an outcome that depends on unpredictable timing of threads sharing data is a race condition. 4. **B** — a single core cannot run tasks truly simultaneously; it time-slices between them. 5. **B** — offloading the download keeps the UI responsive while it runs. 6. **B** — mutual waiting on each other's held resources is deadlock.

Section B (9 marks)

7. **[3]** The regional totals can be computed concurrently because each region's figure is independent of the others (no shared data) (1). Combining them into a grand total must run **after** all regional totals exist (1), because it depends on (uses the output of) those totals — a data dependency forces it to be sequential (1).

8. **[2]** Benefit (1): images load in parallel so the page appears/completes faster and the page stays responsive while they load. Trade-off (1): harder to program/debug, more overhead, and on a single core it is time-sliced rather than truly parallel so the gain may be limited; results may need synchronising. Must be applied to the image-loading scenario.

9. **[2]** Any one reason (2, or 1+1): on a single core there is no true parallelism — just time-slicing — so no real speed-up (1); there is overhead in creating/managing threads and synchronising/recombining results (1); some problems are inherently sequential because later steps depend on earlier outputs.

10. **[2]** A race condition is where the result depends on the unpredictable order/timing of concurrent tasks that share data (1). Example (1): two customers' processes both read "1 seat left" at the same moment and both book it, so the same seat is sold twice.

Section C (5 marks) 11. **[5]** Up to 5 from: Tasks (a) generate-thumbnails, (b) scan-content and (c) save-original are largely independent of each other (each works from the uploaded image), so they could run concurrently (1). A sensible dependency may be noted — e.g. the app may need a valid/safe image before it is *published*, so the content scan result may have to be checked before the photo is shown, even if the work runs in parallel (1). Benefit (1): doing them at once makes the upload feel faster / keeps the app responsive and uses multiple cores. Trade-off (1): concurrency is harder to design and debug, risks race conditions if they touch shared storage, and results may need synchronising; on limited hardware it may not be truly faster (1). Must apply to the upload scenario and weigh both benefit and trade-off.

Total: 6 + 9 + 5 = 20